```
Number>
      <itemOriginalPath>c:\some-
folder\someimagefile.jpg</itemO-
riginalPath>
      <itemNameOnly> itemName-
Only UNDEFINED (9)</itemNameOnly>

<itemCaption>someimagefile.jpg</
itemCaption>
      <itemSize>itemSize UNDE-
FINED (9)</itemSize>
  </image>
```

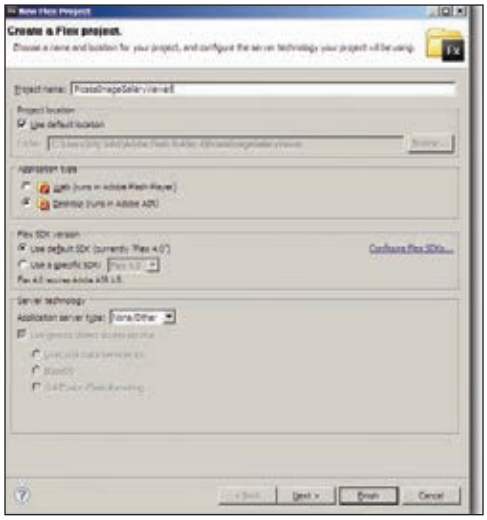
In all of this we only need `<itemThumbnailImage>`, `<itemLargeImage>`, and `<itemName>`. We can use the caption as well, but for our simple example we will avoid it.

Our Flex application needs only to load this XML file and use the list of images from it to populate the gallery. Simple! The execution is even simpler. For the most part this can be done entirely in declarative code. What this means is that we can do most of the coding in MXML – which is similar to XML / HTML. Declarative programming means that we simply declare all the user interface elements, and define some relations and the application works as expected instead of actually coding everything. You can read up on our previous article at www.thinkdigit.com/d/adobefc for a better idea.

In fact, a functional gallery – which simply displays thumbnails – can be created entirely in declerative MXML code, the only ActionScript code we will require is for popping up the image when we click on its thumbnail in the gallery, and even that is minimal.

STEP 1: First, you'll need to create a new Flash Builder project (you can either create either a web project, or you can create a Desktop project that will run in Adobe AIR. Although we're creating a Desktop project, since we aren't using any desktop specific features, you can use the same code for a web project.

STEP 2: From the folder where you have exported the gallery to Flex, copy all the files and folders – there should be an index.xml file, and an images and thumbnails folder – to the "src" folder in you Flex project. Your Flex project will probably be in `C:\Users\<username>\Adobe Flash Builder 4\<project name>\` (or `C:\Users\<username>\Adobe Flash Builder 4 Plug-in\<project name>\` if you have the Flash Builder Eclipse plugin installed). You can also drag and drop these files into the Package Explorer view in Flash Builder.



Starting a new Flex project

STEP 3: First of all we need to load the XML index into our application. We do this using the XML tag in MXML. Add the following inside the `<fx:Declarations>` tag:

```
  <fx:XML id="indexXML"
source="index.xml" />
```

Here we are declaring an XML object called indexXML, and using the file "index.xml" as a source. For our list image gallery though we will need a list of all images from this.

STEP 4: We will now extract a list of all images from this XML file. With support for E4X in ActionScript and Flex, this is very simple. We have our XML object indexXML, we can directly access the "images" tag inside this as: `indexXML.images`. If we access indexXML.images. image it will return a list of all image elements inside the images tag. We can directly pull this into an XMLListCollection, which can be directly used in the list we will add in the next step.

STEP 5: Inside the `<s:Application>` – if you created a web project – or the `<s:WindowedApplication>` – if you created a Desktop application – add the List element.

```
  <s:List id="imagesList"
dataProvider="{index}"
            width="100%"
height="100%" >
      <s:layout>
            <s:TileLayout />
      </s:layout>
  </s:List>
```

This creates a list inside our application which has an id / name imagesList, and fills up to take 100 per cent of the width and 100 per cent of the height of the application. The `<s:TileLayout />` tag inside the `<s:layout>` tag specifies that the list will display content in a tiled layout, as rows and columns of images.

The `dataProvider="{index}"` bit is telling the list to use the `index` object we defined earlier as the source for populating this list. The list will automatically create one list item for each image in the index. It will automatically handle displaying and scrolling in the list. Right now however the list is not geared to handle images, it will simply display information about each image as a text.

STEP 6: What we need to add to this list, is an item renderer. As the name suggests, an item renderer defines how each item in the list is to be rendered / displayed. We will add the following inside the list declaration:

```
  <s:itemRenderer>
      <fx:Component>
            <mx:Image
source="{data.itemThumb-
nailImage}" />
      </fx:Component>
  </s:itemRenderer>
```

We're defining the image component as the item renderer for the list.

The list passes the data associated with each list item, i.e. the contents of the `<image>` tag to the each item in the list as the `data` object. The image to be used for each list item is then accessible through `data.itemThumbnailImage`. This property as you can see from the XML format of the index file, has the releative path to the thumbnail of the image. After adding